

TASK DIVISION — 2 DEVELOPERS [REV 2]

Project: ZeroSixtyThree + 063 Society Website | Supersedes Task Division v1. Read together with the Website Development & Partnership Proposal, the Milestone 1–4 Development Plan, and Developer 1's **plan.md**.

What changed in Rev 2 (why this revision exists)

- The site is an **enhancement of a live site**; the client wants a **maximalist** result with heavy, purposeful motion, and two moods under one roof: **063 Productions = rugged**, **063 Society = elegant**.
- **Milestone 1:** Developer 1's foundation now explicitly includes the **dual-brand design-token system** (rugged + elegant) **and a shared motion foundation** (Framer Motion + Lenis, reduced-motion policy, shared animated primitives). Developer 2's animation task becomes **composing** from those primitives. Loads stay equal (1H / 2M / 2E).
- **Milestone 2:** Developer 2's page builds now require the **maximalist treatment**, the correct **data-brand mode** per page, and **lite-YouTube embeds** for the portfolio gallery. Loads unchanged.
- **Milestone 4:** Deployment now **connects the existing domain zerosixtythree.com** — this **supersedes v1's** "Vercel URL only" note, per the Milestone Plan and the client call.
- **Shared rules:** added — data-brand per page; verify reduced-motion; hold the performance budget.

How to read this document

- The project has 4 milestones. Each has a Start order, a task table per developer, and client dependencies.
- Task levels: **Hard** = complex logic / affects the whole site / risky if done wrong; **Medium** = real skill and time on a known pattern; **Easy** = straightforward placement, wording or checking.
- In every milestone both developers get the same load: **1 Hard, 2 Medium, 2 Easy each**, and each owns one complete feature or flow.

Who does what (short version)

- **Developer 1** — client contact; project base, **dual-brand design system + motion foundation**; main content pages; 063 Society page; booking form; Firebase; email notifications; footer credit banner; deployment.
- **Developer 2** — responsive layout; **composing page animations from Developer 1's motion primitives**; Home, Portfolio and Collaborations pages; the whole admin side and the partnership computation; SEO.

Do they work at the same time?

- Yes, most of the time — both developers work in parallel inside the same milestone.
- Only foundation goes first. In M1, Developer 1 sets up the project base, **design tokens and the motion primitive library** before Developer 2 builds on top. In M3, Developer 1 creates the Firebase structure before Developer 2 connects the admin side to real data.
- Deployment is always the last step of the project.

Project rules for this build

Client setup

- Only Developer 1 contacts the client and collects all credentials, emails, numbers, categories, content and access.
- Developer 2 receives the final values from Developer 1.
- Credentials to collect: SMTP details for Nodemailer (host, port, user, password, sender), the notification email address, Firebase account access, Meta Business access, the client GitHub account, and **domain/registrar access**.

Deployment [updated in Rev 2]

- Deployment target is **Vercel**. Push and publish on Vercel.
- **Connect the existing domain zerosixtythree.com** in Milestone 4 (repoint DNS from the old site). This supersedes the v1 "no custom domain this phase" note.
- Deployment is done last, after every other Milestone 4 task is finished.

Git workflow

- Each developer works on their own branch. No direct push to main.
- Who merges into main is agreed case by case, before the merge.
- Never commit credentials — SMTP and Firebase values go in environment variables.

MILESTONE 1 — Foundation, Design System, Motion System & Landing Page

Goal: Approve the design + motion direction and deliver the landing page, the six-page navigation, the mobile-friendly base, and the reusable foundation (dual-brand tokens + shared motion primitives) that every later page is built on.

Start order

- Developer 1 starts first. The project base, folder structure, **design tokens (both brand modes)** and the **shared motion primitive library** must exist before anything is built on top.
- Developer 2 joins once the base project, the design tokens and the motion primitives are merged into main (usually within the first working day).
- After that, both work at the same time on separate sections.

Also in this milestone: Developer 1 provides the "Developed by" footer credit banner asset (our logo, links to our site) for the shared footer.

Developer 1	LEVEL
Set up the Next.js + Tailwind base project, folder structure, and the dual-brand design tokens (productions/rugged + society/elegant — theme colors, fonts, spacing) from the approved logos, plus the shared motion foundation : Framer Motion + Lenis, the prefers-reduced-motion policy, and the shared animated primitives (Reveal, Marquee, Parallax, KineticHeading, MagneticButton, CustomCursor, PageTransition)	HARD
Build the landing page hero and top sections — maximalist and kinetic — replacing the long-scrolling structure	MEDIUM
Build the navigation bar for the six pages, including the animated mobile menu	MEDIUM
Collect the logo files, theme colors and sample photos from the client, then send them to Developer 2 (and provide the footer credit-banner asset)	EASY
Place the placeholder photos and sample wording in the landing sections	EASY

Developer 2	LEVEL
Build the responsive layout system and breakpoints (on the shared tokens) so the landing page works on phone, tablet and desktop	HARD
Build the lower landing sections: services preview, about preview, call to action, footer	MEDIUM
Compose the landing scroll choreography (reveals, parallax, marquee, smooth scrolling) using Developer 1's shared motion primitives — do not introduce a separate animation approach	MEDIUM
Build the shared small components on the tokens: buttons, section headings, dividers	EASY
Check the landing page on phone/tablet/desktop, fix spacing, and verify the reduced-motion path	EASY

Together

- One short review call with management to approve the layout, colors, fonts and motion direction.

Waiting on the client

- Logo files for ZeroSixtyThree and 063 Society (PDF or high resolution).
- Preferred theme colors for both brands (confirmed: lime #ADFF2A / white / black).
- A few sample photos for the visual direction.
- Blocked until received: the design-tokens task and the placeholder-content task.

MILESTONE 2 — Main Pages, 063 Society Page & Collaborations

Goal: Deliver Home, About, Services, Portfolio/Testimonials, Contact, the elegant 063 Society page, and the Collaborations page — with final content, the correct brand mode per page, and the maximalist treatment throughout.

Start order

- Both developers start at the same time. The pages are independent of each other.
- Developer 1 sends collected content to Developer 2 as soon as it arrives, so Developer 2 is not blocked.
- The YouTube video links must be ready before Developer 2 finishes the portfolio gallery.
- Every page must declare the correct **data-brand** mode: **society** (elegant) for the 063 Society page, **productions** (rugged/maximalist) for the rest.

Developer 1	LEVEL
Build the full 063 Society page in elegant mode (data-brand="society"): sections, service categories and its own content layout, elegant serif treatment and restrained motion — proving the rugged↔elegant alignment	HARD
Build the About page with the final brand story and photos	MEDIUM
Build the Services page (weddings, corporate events, event program support, music & entertainment, AV/production) as an animated bento layout	MEDIUM
Collect the final service descriptions, testimonials, partner logos and portfolio photos from the client, then send them to Developer 2	EASY
Run the YouTube upload session with management on the developer computer and send the video links to Developer 2	EASY

Developer 2	LEVEL
Build the Portfolio/Testimonials page: the maximalist photo gallery (layered / pinned) and the YouTube video sections using performant lite-YouTube embeds , composed from Developer 1's motion primitives	HARD
Build the Home page final sections with the real content — maximalist and kinetic (productions mode)	MEDIUM
Build the Collaborations page: partner-logo marquee and the linked project photos (bulletin-board feel)	MEDIUM
Build the Contact page layout and the contact-details block in the correct brand mode (the booking form comes in Milestone 3)	EASY
Connect all six navigation links to the finished pages, verify each opens correctly, and check the reduced-motion path	EASY

Together

- One review call with management to approve all pages before Milestone 3 starts.

Waiting on the client

- Final service descriptions for ZeroSixtyThree and 063 Society.
- Portfolio photos, videos and testimonials.
- Collaboration/partner logos and the projects shown under each.
- YouTube account login, entered by management themselves.
- Blocked until received: the About, Services, Home, Collaborations and Portfolio content tasks.

MILESTONE 3 — Booking Form, Firebase Database & Admin Side

Goal: Deliver the inquiry/booking form, the Firebase database, email notifications, and the secure admin side with the partnership computation.

Start order

- Developer 1 starts first with the Firebase structure and the security rules. The admin side cannot read real records before this exists.
- Developer 2 can start immediately on the admin login and the records layout using sample data, then connects to the real database once the Firebase structure is merged into main.
- The 2% / cap logic lives in a **shared partnership-calc helper** (lib/utils) so the form, admin and invoice use one source of truth.

Developer 1 — booking & inquiry flow	LEVEL
Set up Firebase: database structure for customer, booking and payment-status records, plus the security rules (public may create a validated inquiry; only the authenticated admin can read/manage)	HARD
Build the booking form with all approved fields: name, email, mobile, event type, date, location, services, guests, notes, and the source of inquiry (063 or 063 Society)	MEDIUM
Build the email notification for new inquiries using Nodemailer and the client SMTP details	MEDIUM
Add the form validation messages, the thank-you state and the error state	EASY
Collect the SMTP details, notification email, Firebase editor invitation and final field list from the client, then send Developer 2 the values needed for the admin side	EASY

Developer 2 — admin side	LEVEL
Build the 2% partnership computation with the AED 250 cap per booking, the monthly summary of paid customers, and the automatic invoice/billing statement (using the shared partnership-calc helper)	HARD
Build the secure admin login and the separate admin link that is not shown on the public website	MEDIUM
Build the records screen with viewing, searching and filtering of bookings	MEDIUM
Add the config-driven booking status control (New/Confirmed/Paid/Cancelled/Refunded) and the actual-amount-collected field	EASY
Add the Returning Customer tag and the indicator for whether the customer originally came from the website	EASY

Together

- One review call with management to test the form and the admin side with sample entries.

Waiting on the client

- Firebase account created by management, plus the editor invitation for the developer.
- SMTP details for Nodemailer and the email address that receives new inquiries.
- Final list of fields for the booking form and the admin side.
- Blocked until received: the Firebase setup task, the email-notification task and the final field list in the booking form.

MILESTONE 4 — SEO, Tracking, Deployment, Testing & Handover

Goal: Publish on Vercel and connect the existing domain, set up SEO and Meta Pixel, transfer the source code to the client GitHub, test everything and hand over.

Start order

- Both developers start at the same time on SEO, Meta Pixel and the small finishing tasks.
- Deployment comes last. Developer 1 publishes to Vercel only after Developer 2 has merged the SEO work and both branches are clean.
- Live testing and the handover walkthrough happen after the site is published.
- **Rev 2:** the existing domain **zerosixtythree.com** is **connected** in this phase (DNS repointed from the old site).

Developer 1	LEVEL
Deploy to Vercel (production build; environment variables for Firebase and SMTP) and connect the existing domain zerosixtythree.com — repoint DNS from the old Canva-hosted site. Done last.	HARD
Set up the Meta Pixel and confirm that page-view and form-submit events are recorded	MEDIUM
Transfer the complete source code to the client GitHub account and confirm access	MEDIUM
Add the favicon, browser tab titles and the social share image	EASY
Test the full booking flow on the live URL: submit, save and email notification	EASY

Developer 2	LEVEL
Full SEO setup: page titles and descriptions for every page, sitemap, robots file, and page-speed fixes	HARD
Test the admin side on the live site: login, status changes, the 2% computation with the cap, and the invoice	MEDIUM
Prepare the short admin walkthrough guide for management	MEDIUM
Check every page, link, image and video on the live site	EASY
Check the live site display on phone, tablet and desktop	EASY

Together

- Agree who merges the final release branch into main.
- Handover walkthrough call with management, including the admin-side demo.

Waiting on the client

- Meta Business access for the Meta Pixel setup.
- Client GitHub account for the source-code transfer.
- **Domain / registrar access** for connecting zerosixtythree.com.
- Blocked until received: the Meta Pixel task, the GitHub transfer task and the domain connection.

Simple rules for both developers

- Work only on your own branch. Never push directly to main.
- Agree who merges into main before every merge. Pull the latest main before you start a new task.
- Only Developer 1 contacts the client. Developer 2 asks Developer 1 for any value, credential or content.
- Never commit credentials. SMTP and Firebase values go in environment variables.
- **Use the correct data-brand mode per page** — *society* (elegant) for the 063 Society page, *productions* (rugged/maximalist) elsewhere. Compose motion from the shared primitives; do not hand-roll one-off animations.
- **Verify the reduced-motion path** (prefers-reduced-motion) on anything you animate, and **hold the performance budget** (optimized images, lite/lazy video, code-split heavy motion). Maximalism must not mean slow.
- There is no QA team. You test your own work before you call it done.
- Do not review or correct the other developer's tasks. Each task has one owner.
- Finish your milestone tasks before starting anything from the next milestone. Deployment is always the last step.

When is a task done?

- It works on phone, tablet and desktop, and the reduced-motion path is clean.
- You tested it yourself and it does what the milestone says.
- There are no console errors and no broken links or images.
- The code is pushed to your branch and merged into main after the agreed merge.
- For client-facing tasks: management reviewed it, and any revisions inside the approved scope are already done.

Fair and open work

- Both developers receive the same document. There are no separate or hidden instructions.
- Every task has one visible owner. Nothing is assigned in private.
- Every milestone gives each developer the same number of Hard, Medium and Easy tasks (1 / 2 / 2).
- If a task turns out to be harder than its label, say so early and trade it for a task of the same level. The balance stays equal.

If a new developer joins

- Give them this document (Rev 2) first — it is the full picture of the project.
- Developer 1 gives them repository access, branch rules and the environment values they need. They do not contact the client.
- Re-split the current milestone so all developers still get the same number of Hard, Medium and Easy tasks.
- Give them one complete feature or flow to own, not scattered small tasks. They open their own branch and follow the same merge rule.